



Python Jackfruit Project

Topic:Expense Manager

Presented by

1. S.Adhvik

SRN:PES1UG25CS433

2. Preetam R C

SRN:PES1UG25AM291

3. Rohitsamartha B

SRN:PES1UG25AM321

4. Rohan C H

SRN:PES1UG25EC212

Acknowledgement:

We sincerely express our gratitude to **Teja I** ma'am for valuable guidance, support and encouragement throughout this project. Their suggestions greatly helped us in completing this project successfully.

Expense Manager Project: Technical Report

Executive Summary

This report documents the development of a comprehensive Expense Manager application built using Python and Tkinter GUI framework. The project demonstrates practical implementation of object-oriented programming, data persistence using CSV files, and multi-screen application design. The application allows users to add, view, delete, and analyze personal expenses with category-wise breakdown and graphical visualization.

1. Introduction

1.1 Project Overview

The Expense Manager is a desktop application designed to help users track their daily expenses efficiently. The application provides a user-friendly interface with multiple screens for adding new expenses, viewing all recorded expenses, analyzing spending patterns, and calculating monthly savings. It implements a modular architecture with separate backend and frontend components, making the code maintainable and scalable.

1.2 Project Objectives

The main objectives of this project were:

- Create a user-friendly GUI application using Tkinter framework
 - Implement persistent data storage using CSV file format
 - Develop category-based expense tracking with dropdown menus
 - Provide analytical features including savings calculation and pie chart visualization
 - Maintain clean code structure using Object-Oriented Programming principles
 - Implement a multi-screen navigation system for better user experience
-

2. Architectural Approaches and Design Decisions

2.1 Modular Architecture

The application follows a **Model-View-Controller (MVC) pattern** with the following separation:

Backend Module (`backend.py`) contains all data handling logic and is completely independent of the user interface. This separation provides advantages including data independence, CSV file management, and core functions like `initialize_file()`, `save_expense()`, `load_expenses()`, and `delete_expense()`.

Frontend UI Modules are further divided into specialized frames: `main.py` (Controller) manages frame switching and application flow, `add_ui.py` (AddExpenseFrame) handles expense entry with category dropdown, and `view_ui.py` (ViewExpenseFrame) displays expenses and provides deletion and analysis features.

2.2 Data Persistence Strategy

The application uses CSV (Comma-Separated Values) format for storing expenses with four columns: Date, Item, Amount, and Category. CSV provides advantages including simple format, human-readable structure, no database setup required, and portability. The CSV structure allows each expense entry to be a new row in the file for easy management.

2.3 Category Dropdown Implementation

Initially, category input was planned as a text entry field, but was changed to use a **ttk.Combobox** dropdown menu to prevent invalid category entries, provide consistent categorization, and improve user experience with pre-defined options: Rent, Food, Transportation, Fees, Personal, Medicines, Clothes, and Other.

2.4 Expense Analysis Approach

The ViewExpenseFrame implements two analysis methods: **Savings Calculation** compares user income with total expenses to calculate net savings, and **Pie Chart Visualization** uses Matplotlib to show percentage distribution of expenses by category. The pie chart dynamically aggregates expenses by category and visualizes spending patterns, helping users identify high-spending categories.

3. Challenges Faced and Solutions Implemented

3.1 CSV File Operations & Multi-Screen Navigation

CSV File Handling: The challenge of managing file creation, reading, and deletion while preventing corruption was solved by implementing `initialize_file()` to check file existence, using append mode ("a") for adding expenses, and employing a read-modify-write approach for deletions. `load_expenses()` skips the header row using `next(reader)` to avoid processing headers as data, and empty line validation ensures corrupted rows don't get processed.

Multi-Screen Navigation: Managing multiple screens without creating memory leaks or slow transitions was achieved using the stacking frame pattern where all frames are created at startup. The `show_frame()` method in `ExpenseManagerApp` switches frames by raising them to the front using `tkraise()`. This pattern allows frames to be raised without recreating them, providing efficient memory usage.

3.2 Data Consistency & Input Validation

Data Consistency: The Treeview widget shows outdated data after modifications because it doesn't auto-refresh. The solution implemented `load_data()` method that clears all rows and reloads from backend after every modification. The `tkraise()` method in `ViewExpenseFrame` was overridden to auto-load data when the frame becomes visible.

Input Validation: Users entering invalid data (non-numeric amounts, empty fields) posed a crash risk. The solution added field validation in `submit_expense()` to check if all fields are filled before saving, used `.isdigit()` method for numeric input validation, and employed message boxes for user-friendly error feedback. Type conversion with `float()` handles precise calculations with error handling.

3.3 Category Dropdown & Pie Chart Visualization

Category Dropdown: Initially using Entry widget allowed arbitrary text entry, leading to inconsistent categorization. The solution replaced `tk.Entry` with `ttk.Combobox` (dropdown menu) set to `state="readonly"` to prevent manual text entry. Pre-defined categories are stored in the form with `.current(0)` setting the default category on startup.

Pie Chart Visualization: Generating meaningful visualizations from individual transactions required aggregating data by category. The solution created `category_totals` dictionary to aggregate expenses by category, checked if expenses exist before plotting to prevent crashes, and used Matplotlib's `plt.pie()` with percentage labels for meaningful visualization. Graph displays both category names and percentage distribution.

3.4 Savings Calculation

Problem & Solution: Calculating net savings required handling type conversions and ensuring accurate arithmetic. Income input (string) needed conversion to float, expense amounts stored as strings in CSV required conversion during calculation loop, and negative savings needed distinct visual indication. The solution validated income input using `.isdigit()` before conversion, converted all amounts to float during the calculation loop, and applied conditional formatting with green text for positive savings and red text for negative savings to provide clear user feedback.

References

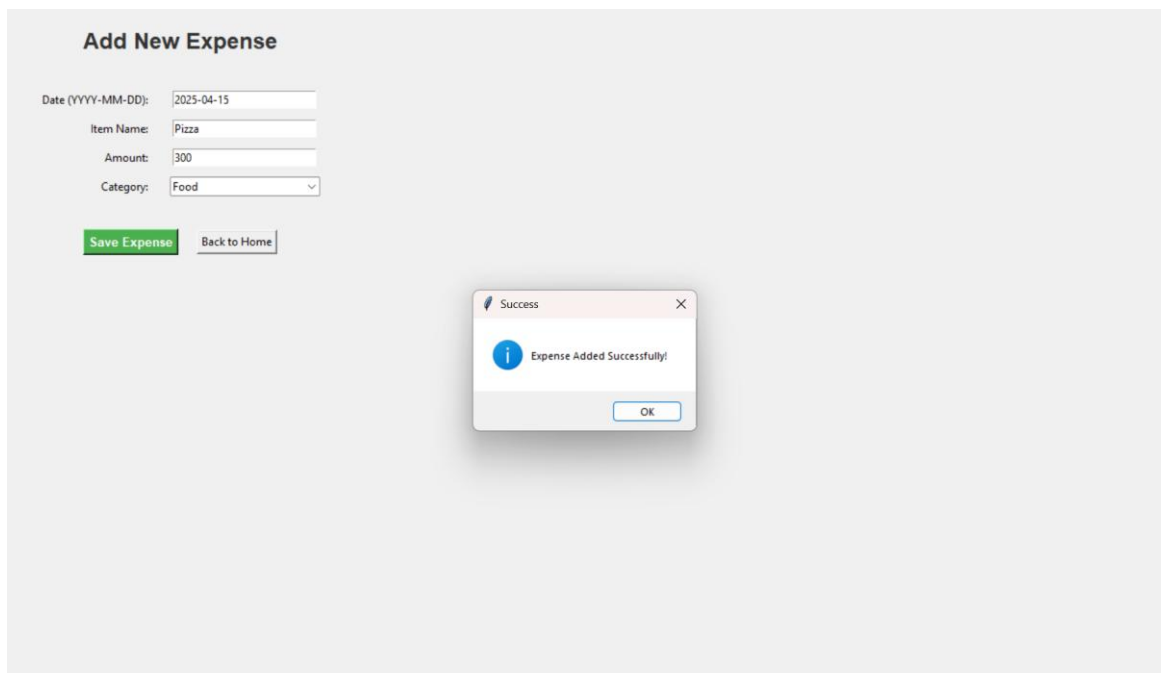
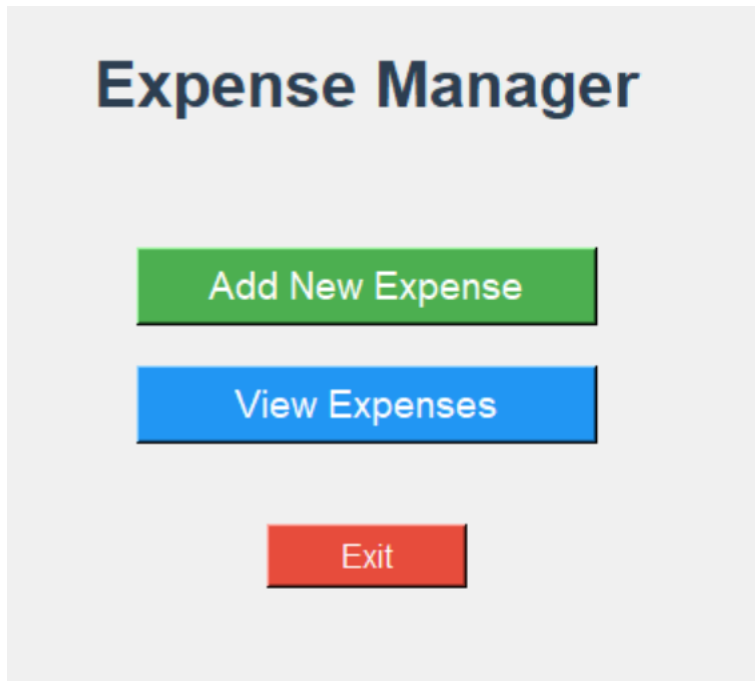
[1] Python Software Foundation. (2024). Tkinter – Python interface to Tcl/Tk. Python Documentation.

[2] Python Software Foundation. (2024). csv – CSV File Reading and Writing. Python Documentation.

[3] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.

[4] Python Software Foundation. (2024). os – Miscellaneous operating system interfaces. Python Documentation.

Sample Output:



Manage & Analyze Expenses

Date	Item	Amount	Category
2025-04-15	Pizza	300	Food
2025-09-30	House rent	10000	Rent
2025-10-18	bus	50	Transportation
2025-12-31	Party	20000	Personal
2025-12-13	College	22190	Fees

Refresh ListDelete Selected

Annual Analysis

Enter Annual Income:

Calculate Savings

Total Expense: 52540.0Net Savings: 947460.0

Show Expense Chart

Back to Home

Expense Distribution by Category

